CAREERVAULT · FINAL YEAR EVALUATION · 2 NOV 2026

Evaluation demo guide: the five security points

The second review gave us five security points. This page explains what we built for each one in simple words, what to say, and exactly what to click. The screenshots are from a full run on 31 Oct. The live site behaves the same once it has been reseeded (see Before you start).

Website career-vault-steel.vercel.app/demo Slides **CareerVault - Security Review Response.pptx**

Blockchain **Polygon Amoy (now optional, a badge)**

Summary · Where we differed · Before you start · 1 · Signing key · 2 · Make a document · 3 · Verify it · 4 · Chain badge · 5 · SHA-256 lab · 6 · Attacks · 7 · Database · Questions · Problems to avoid

---

SHORT ANSWER

All 5 points are built, tested and deployed

Each point is in the code, covered by automated tests (458 pass), and live on the website. Use the words in the last column.

POINT	DONE?	WHAT WE BUILT (SIMPLE WORDS)	WHAT TO SAY
1 · Employee key Parts 1–2	● Done	Every manager and HR person makes a **personal signing key** once, from date of birth, a personal ID and 3 security answers. PBKDF2 (600,000 rounds) turns them into a 256-bit key. It is stored only **locked with AES-256-GCM**. At every signature they type the details again: that unlocks the key, which **seals** the document (HMAC-SHA256), and then the company RSA-2048 key signs.	"AES locks the key, SHA-256 (as HMAC) seals the document, RSA-2048 signs. Two people, two personal keys. A stolen login cannot sign."

POINT	DONE?	WHAT WE BUILT (SIMPLE WORDS)	WHAT TO SAY
2 · No proof files Part 3	● Done	The `.jsonld` download and the file checker are gone. A verifier sends only the **Document ID** (or a share link). Our server does every check and answers with the result, the company and who signed. No salt, hash, key or algorithm name is sent back.	"The verifier learns the result, not how to recompute it. A test fails if anything cryptographic ever appears in the reply."
3 · Own bytes + SHA-256 Part 5	● Done	No JSON library any more: we write the document as exact bytes ourselves (tag, length, value). SHA-256 is also written **by hand, bit by bit**, with a page that shows every step for “abc”.	"Production uses the standard SHA-256; the lab is our own copy that proves we understand it, checked live against the browser."
4 · Blockchain optional Part 4	● Done	The verdict depends only on our own checks. The chain is a small **badge**: pending, timestamped (with a PolygonScan link), not confirmed, or unreachable.	"It works as a complete product without a blockchain. Polygon adds a public timestamp."
5 · Input hardening Part 6	● Done	Every request is screened before it reaches our code (no `__proto__`, no `<script>`, no control characters). IDs and tokens have strict formats. All database queries are parameterised. Errors never show table names.	"We don't block SQL words; we make them harmless. `DROP TABLE` as a name is saved as text, and the table is still there."

BE READY FOR THIS

Where we did it differently, and why

Say these confidently. Each one is a security reason, not a shortcut.

SUGGESTION	WHAT WE DID, AND THE REASON
Use the AES-encrypted key as the HMAC key	AES-GCM gives different output every time (random IV), so it would have to be stored; then anyone who reads the database has the HMAC key. So AES **locks** the key, the server unlocks it in memory for one signature, and HMAC uses the **real** key.
Make the key from DOB, ID and answers	Done, but honestly: DOB and ID are not secrets (HR knows them). The strength comes from the random salt, 600,000 PBKDF2 rounds and the server lock. The answers add a check only the employee can pass at each signature.
Strip SQL injection syntax from input	Blocklists break real names (O'Brien) and are easy to get around. Every query is parameterised, IDs have strict formats, and an ESLint rule bans the unsafe raw-SQL functions.
Zod on every API endpoint	The API already checks every field with class-validator (NestJS's standard). Zod stays on the browser forms. Two validators on one endpoint would drift apart.
Hide algorithms and public keys	Removed from the verify page. But security never depends on hiding the algorithm (Kerckhoffs's principle). What must stay secret, the salts and keys, does.
Remove the proof file	Done. It reverses October's “works without our server” point, on purpose: personal keys on the server make an offline check impossible. Trade-off: if CareerVault is down, verification waits.

TEN MINUTES BEFORE

Before you start

The live site is ready (reseeded and anchored on 1 Nov)

- Alice · Experience Letter, issued and anchored: `/verify/8291fc08-d0f4-47a9-b206-7bc1a5482ceb`
- Alice · Salary Proof, issued and anchored: `/verify/46969431-8f7b-4290-8d0c-5657f3f1e737` (holder name hidden on a public lookup)
- Polygon transaction: `0x5121025e…ccc5c758`
- Bob · Experience Letter, signed by Marcus and **waiting for HR**: sign in as Hannah → Approvals for a one-click live approval (step-up only).

- Nothing older exists, so no document on the site fails.

PASSWORD (ALL)	HOLDER	MANAGER
`Password123@` Not Password@123.	`alice@holder.example.com` Requests the document.	`marcus@techcorp.example.com` Signs with step-up.

HR	ORG ADMIN
`hr@techcorp.example.com` Co-signs with step-up.	`admin@techcorp.example.com` Anchor now.

Signing details for the seeded signers (Marcus, Hannah, Gabriel)

- Date of birth `15/01/1990` · Personal ID `DEMO-0001`
- First school `Somaiya` · Birth city `Mumbai` · First pet `Bruno`
- Case and extra spaces don't matter. **5 wrong tries lock signing for 15 minutes**, so do the "wrong answer" demo only once.

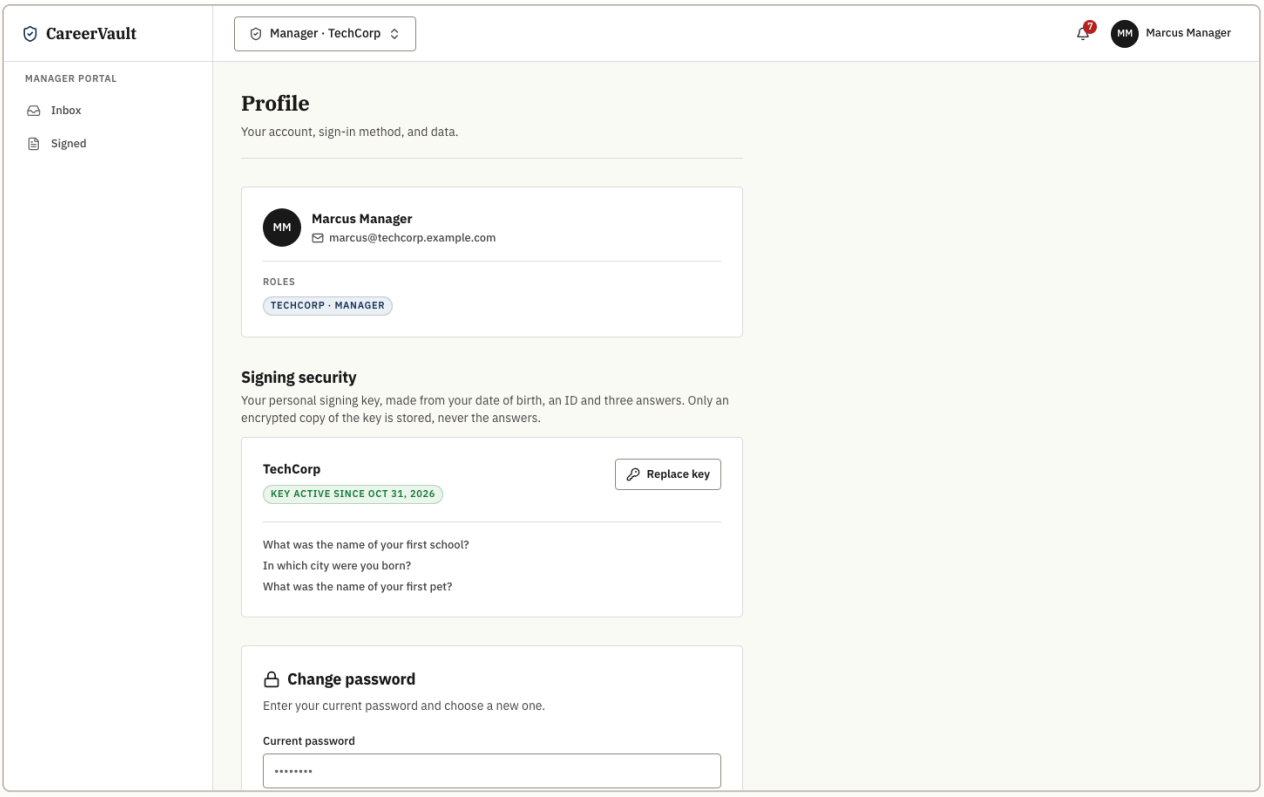
PART 1 about 1 min Point 1

Show the personal signing key

1.1 **Marcus (manager)** `marcus@techcorp.example.com`

1. Sign in, click the name (top right) → **Profile**.
2. Point at **Signing security**: “Key active since …” and the three chosen questions.
3. Optional: **Replace key** shows the form. Replacing asks for the **current** details first.

> **Say:** *The answers became a key with PBKDF2, 600,000 rounds. Only the AES-locked key is stored, never the answers. The screen only knows which questions he picked.*



Profile → Signing security.

PART 2 about 4 min **Point 1**

Make a document live, with two personal keys

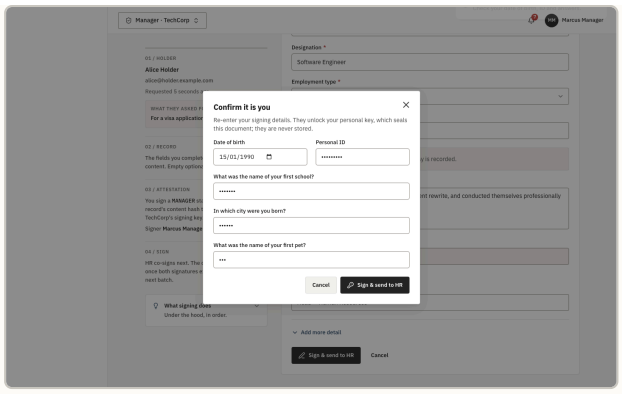
2.1 Alice (employee) `alice@holder.example.com`

1. **Request document** → **Experience Letter** → **TechCorp** → **Review & send**.

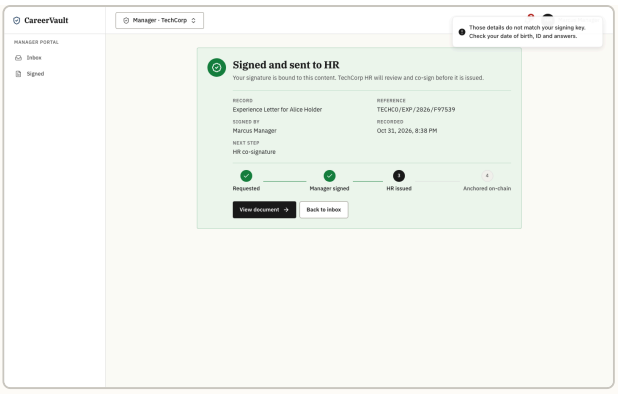
2.2 Marcus (manager)

1. **Inbox** → open the request → **Draft & sign**. Fill Employee code, Designation, Date of joining, Conduct summary.
2. Click **Sign & send to HR**. The **Confirm it is you** dialog opens.
3. Type the details with **one wrong answer** (pet `Rex`) → refused. Fix it → signed.

> **Say:** *Wrong answer: the key does not unlock, nothing is signed, and the attempt is recorded. Right answers: the key seals these exact bytes with HMAC-SHA256, then TechCorp's RSA-2048 key signs.*



The step-up (answers masked).

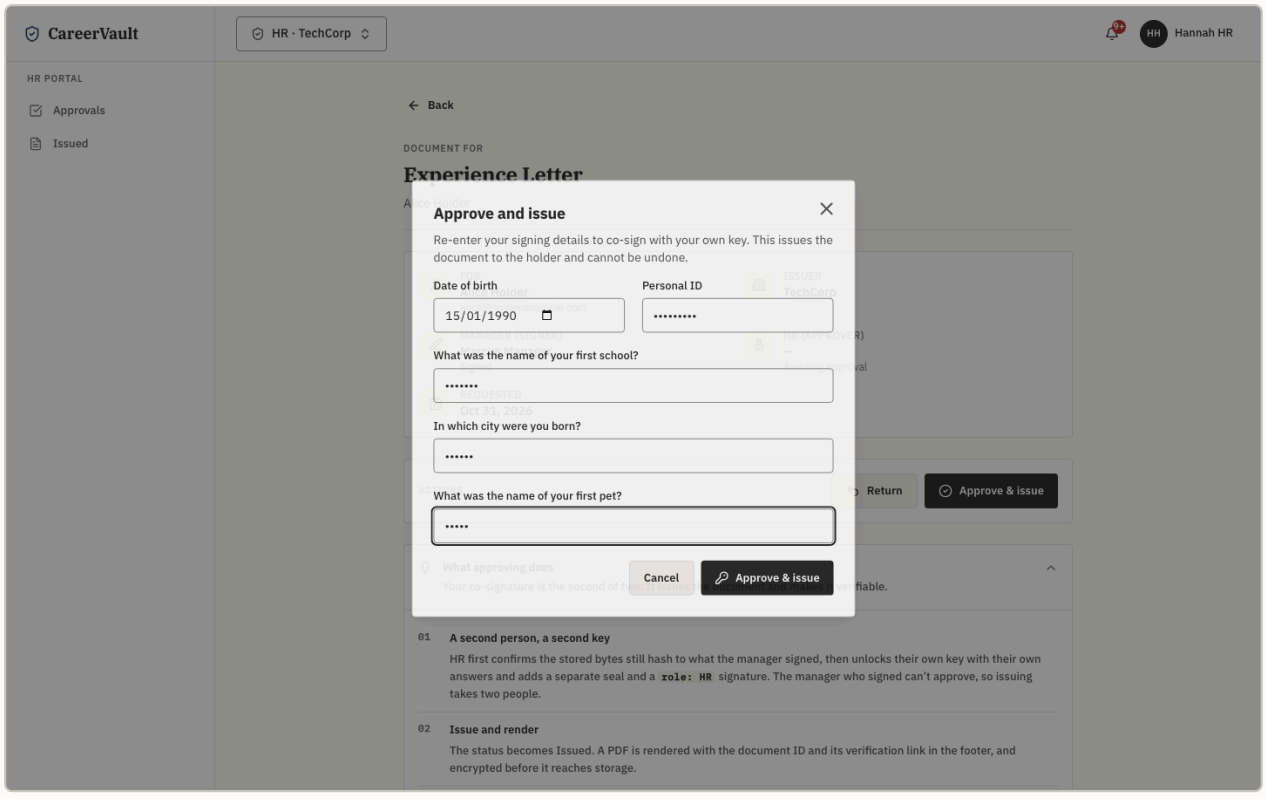


Signed and sent to HR.

2.3 Hannah (HR) `hr@techcorp.example.com`

1. **Approvals** → open it → **Approve & issue** → type **her** details → confirm.

> **Say:** *HR first checks the stored bytes still hash to what the manager signed, then seals with her own key. Two people, two keys, one document.*



HR's own step-up.

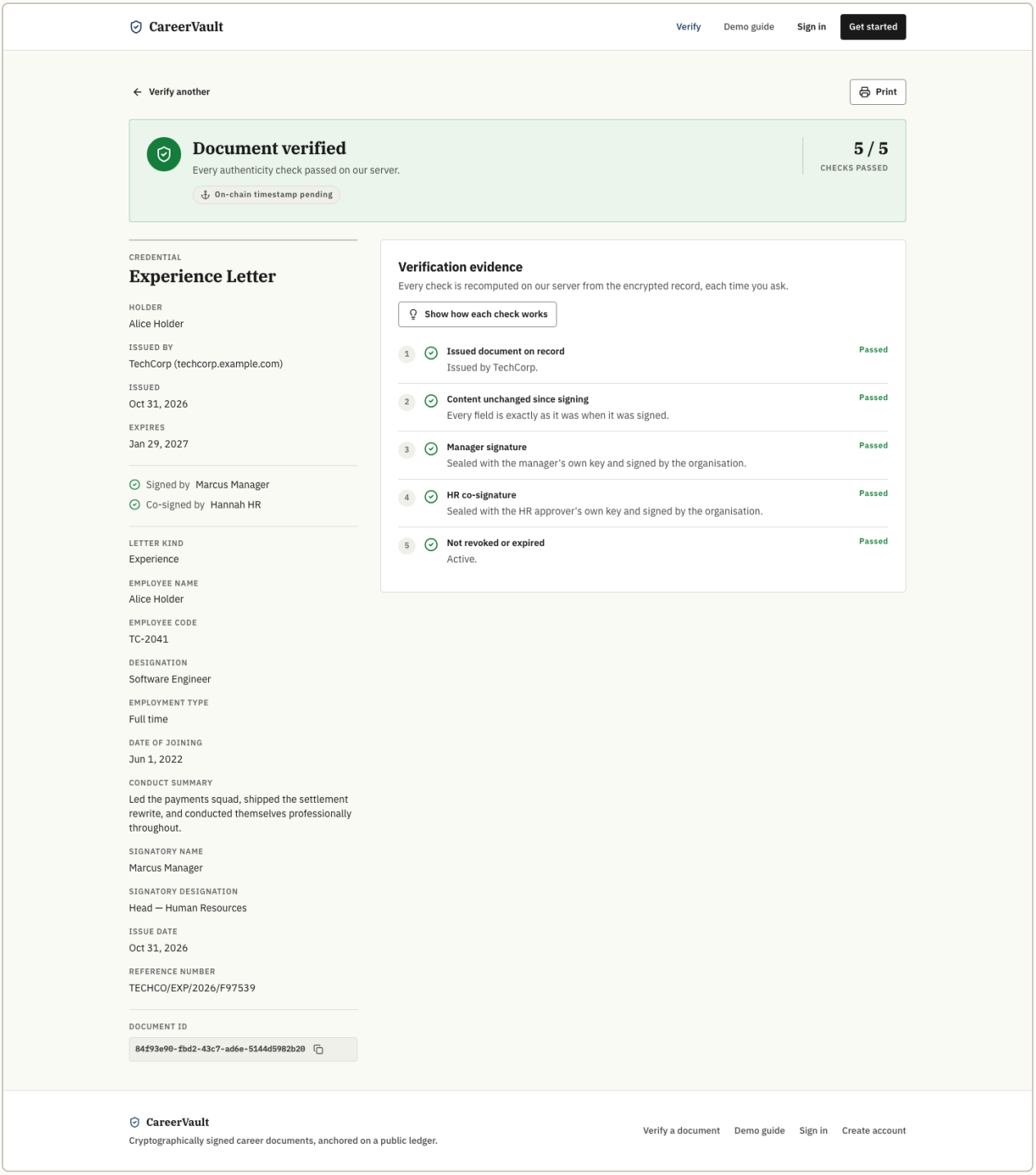
PART 3 about 2 min Point 2

Verify it on our server, by Document ID

3.1 Anyone, no login

1. On the document page click **Open public verification** (or paste the Document ID at `/verify`).
2. Show **Document verified · 5 / 5**, “Signed by Marcus · Co-signed by Hannah”, and the badge **On-chain timestamp pending**.
3. Optional: open DevTools → Network → the `verify/document/…` reply. No hash, salt, key or algorithm in it.

> **Say:** *The browser sent only the ID. The server decrypted the record, rebuilt the bytes and re-checked the hash, both seals and both signatures. It is verified before the blockchain is even involved.*



Verified before anchoring: the chain is only a badge.

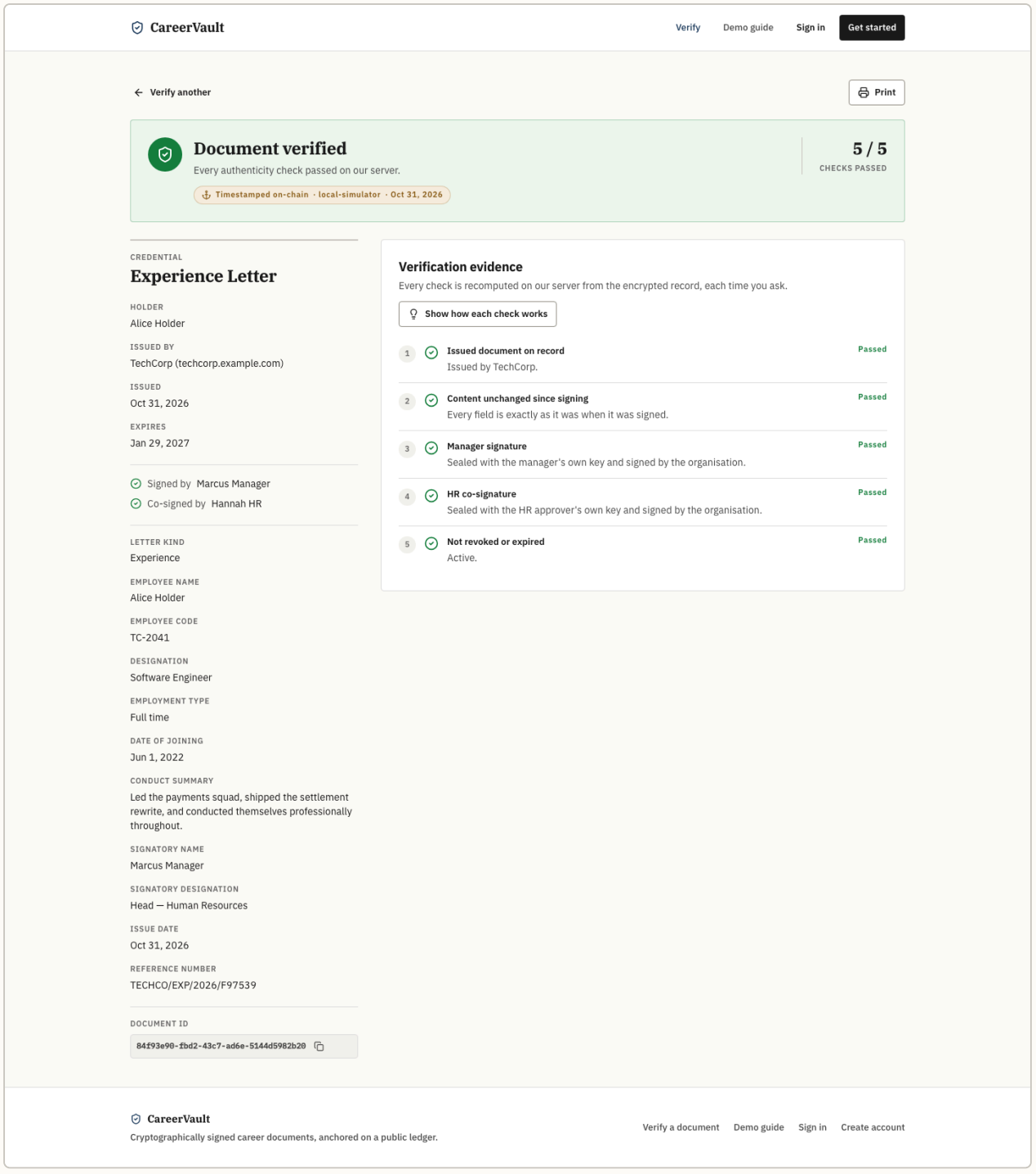
PART 4 about 2 min **Point 4**

Anchor it, and watch only the badge change

4.1 **Olivia (org admin)** `admin@techcorp.example.com`

1. **Analytics** → Blockchain anchoring → **Anchor now**. Open the transaction on PolygonScan.
2. Refresh the verification page: same verdict, badge now **Timestamped on-chain · Polygon Amoy** with **View transaction**.

> **Say:** *The verdict did not change; only the badge did. If Polygon were down, it would still say Verified and the badge would say unreachable.*



After the batch (this capture was the local simulator; the live site shows Polygon Amoy).

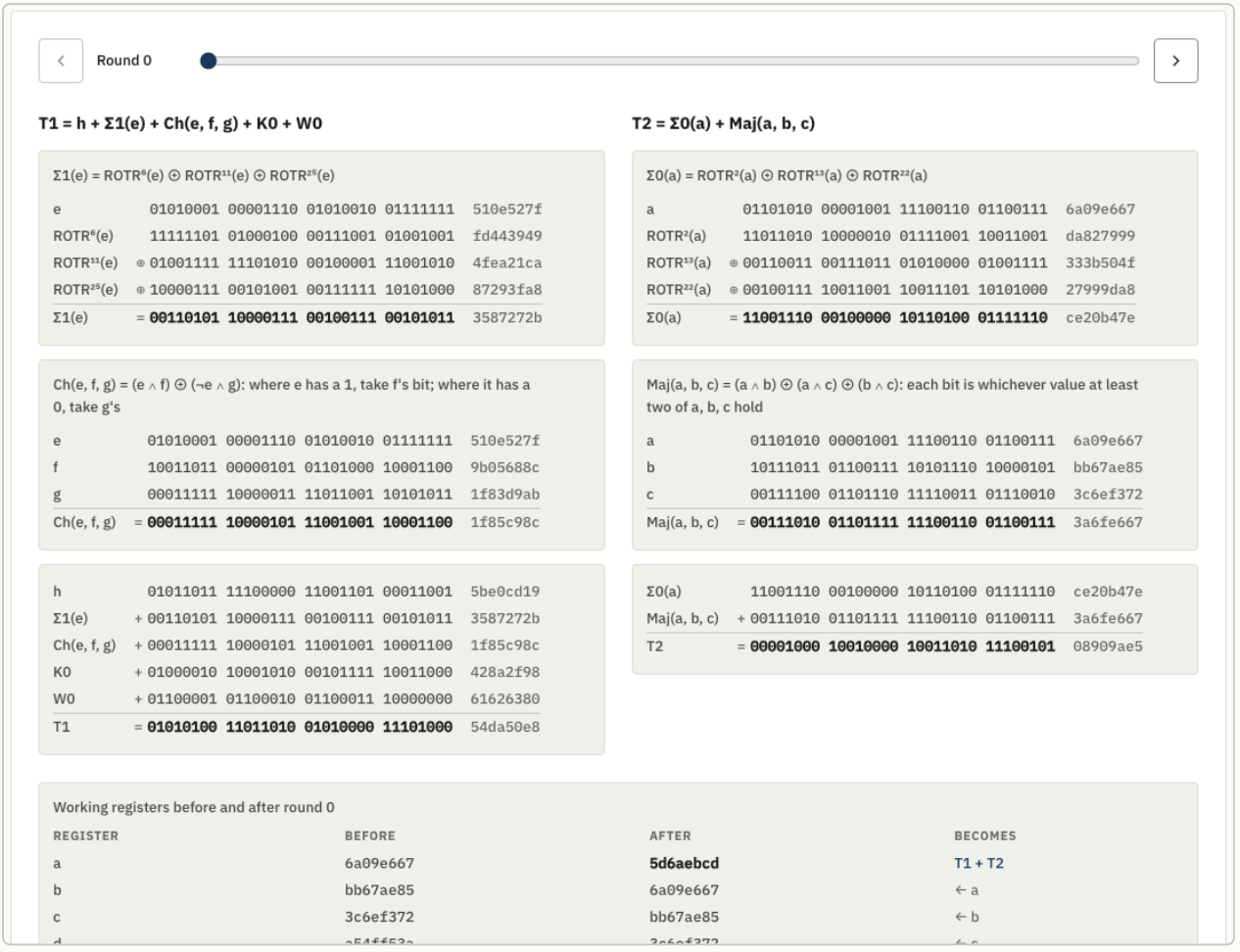
PART 5 about 3 min Point 3

SHA-256, bit by bit

5.1 **No login** /demo/sha256

1. Type `abc`. Section 1: 24 message bits, the single **1** bit, 423 zeros, 64-bit length = 512.
2. Section 2: click row 16. W16 = σ1(W14) + W9 + σ0(W1) + W0, each rotate and XOR shown in binary.
3. Section 3: step through rounds with the arrows; show Ch (“where e is 1 take f, else g”) and Maj (“majority vote”).
4. Section 4: the digest `ba7816bf…20015ad` and the green line: matches the browser's own SHA-256.
5. Type 56+ characters to show the second block.

> **Say:** *This is our own SHA-256 with no library. Its round-0 values match the FIPS 180-4 standard's worked example, and the tests check the official NIST answers.*



Round 0 of “abc” at the bit level.

PART 6 about 2 min **Point 5**

Attack our own API

6.1 **Terminal** live site; all public endpoints

```
B=https://career-vault-steel.vercel.app/api/v1
curl -s -X POST $B/auth/login -H 'content-type: application/json' \
  -d "{\"email\":\"' OR 1=1 --\",\"password\":\"x\"}"
→ 400 "email must be an email"

curl -s -X POST $B/auth/login -H 'content-type: application/json' \
  -d '{"email":"a@b.test","password":"x","__proto__":{"isAdmin":true}}'
→ 400 "The request was refused because it contains a reserved key."

curl -s -X POST $B/auth/register -H 'content-type: application/json' \
  -d '{"email":"x@y.test","password":"Password123@","fullName":"<script>alert(1)</
→ 400 "The request was refused because it contains markup."

curl -s "$B/verify/document/1'%20OR%20'1'='1"
→ 400 "Validation failed (uuid is expected)"
```

> **Say:** *And the stored case: a name like* `Robert'); DROP TABLE users;--` *is saved as plain text, because every query is parameterised. Our automated test registers it and then checks the users table is still there.*

The full suite: `cd server && npm run test:e2e -- test/adversarial` → **14 passed**.

PART 7 about 1 min **Points 1 & 2**

Show the database holds only locked data

7.1 **Supabase → Table Editor** read only, never edit a cell

1. `documents`: `content_json`, `salt`, `manager_seal`, `hr_seal` all start with `cvenc:v1:` (AES-256-GCM).
2. `employee_keys`: `wrapped_key` starts with `ekw1:`, `kdf_iterations` = 600000, `question_ids` only. No answers anywhere.

```
content_json            | salt                      | manager_seal
"cvenc:v1:5906e901d7…   | cvenc:v1:5906e901d74cb8… | cvenc:v1:5906e901d74cb8…

version | kdf_iterations | question_ids                        | wrapped_key
      1 |         600000 | {first-school,birth-city,first-pet} | ekw1:m0enlqqSrj-
```

LIKELY QUESTIONS

Short answers

QUESTION	ANSWER
Why PBKDF2 and not just SHA-256 of the answers?	SHA-256 is fast, so guesses are cheap. PBKDF2 repeats HMAC-SHA256 600,000 times (OWASP's figure), making every guess slow. The random salt means two people with the same answers get different keys.
What is HMAC, and why not plain SHA-256 with the key appended?	HMAC hashes twice with the key mixed in (inner and outer pad). Plain SHA-256(key ‖ data) is open to length-extension; HMAC is the standard keyed hash.
Where is the AES key?	`KMS_MASTER_KEY`, only on the server (Render secret). HKDF derives a separate key per organisation for employee keys, so one key never serves two purposes.
What if someone steals a manager's password?	They can log in but cannot sign: signing needs the step-up answers, and 5 wrong tries lock it for 15 minutes.
What if the manager forgets the answers?	Another org admin resets the key (never their own), the manager is notified, and enrols again. Old documents still verify: keys are retired, never deleted.
Why remove JCS if it is a standard?	JCS existed so outsiders could rebuild the text. Verification is now server-only, so our own fixed byte layout is simpler: no number formatting or escaping edge cases. It is length-prefixed, so fields cannot blur together.
Why is the hash still stored in plain text?	It is a fingerprint, not data: it reveals nothing without the secret salt, and it is the Merkle leaf for the chain.
Is the chain still useful?	Yes, as a public, independent timestamp. It no longer decides validity.
Can a database admin forge a document?	They would need the AES master key to write sealed fields and the employee's answers to make a seal. Strict mode refuses planted plaintext rows.

AVOID THESE

Problems to avoid

Don't trigger the lockout

Only one wrong answer during the demo. After 5 in 15 minutes the signer is locked (HTTP 429); then use another signer or wait.

Old documents

Anything signed before 31 Oct shows **Verification failed** (old hashing). Reseed first, or don't open them.

Render cold start

Open `/demo` 5 minutes early so the server is awake. The first request can take 30–60 seconds.